FAME-QP

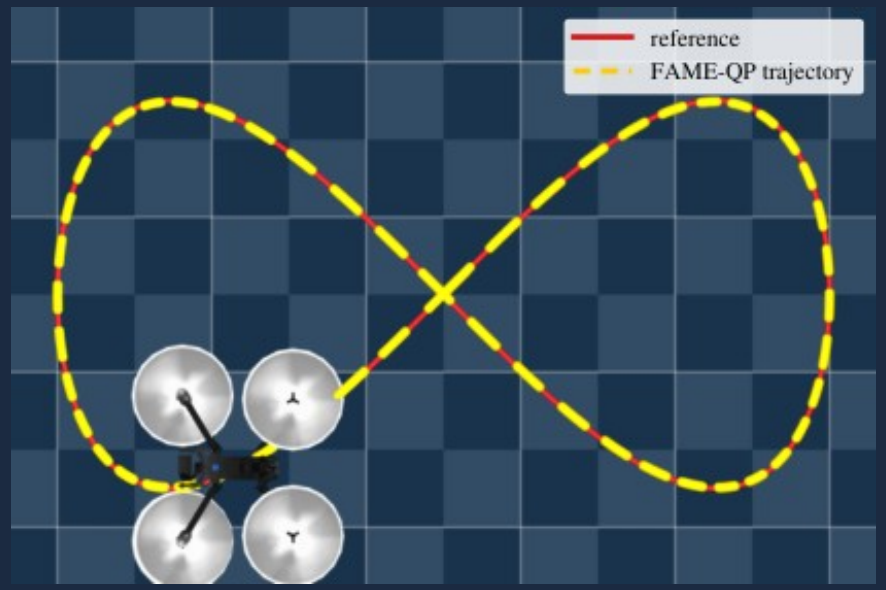
A GPU-Accelerated QP Solver for Real-Time Model Predictive Control

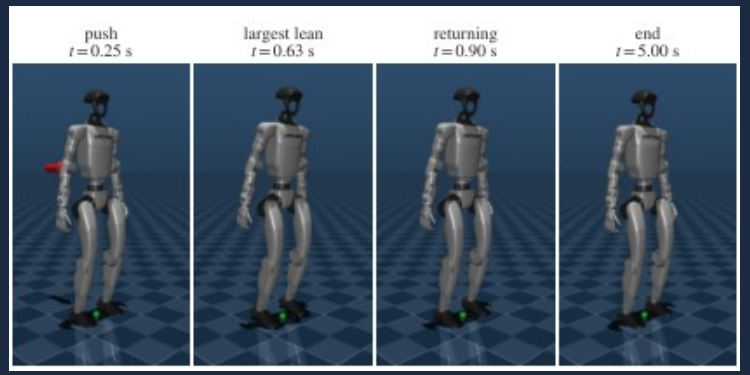
A GPU-Accelerated Quadratic-Programming Solver for Real-Time Model Predictive Control

Santosh Mohan Rajkumar · Debdipta Goswami · Stephanie Stockar
The Ohio State University

arXiv 2611.13766 (2026-11-10) · Open source: github.com/santoshrajkumar/fame-qp

Paper Review





Images: paper Fig. 2 / Fig. 3 (MuJoCo simulation)

One-Sentence Summary

Same algorithm; ADMM's GPU execution overheads cut one by one

FAME-QP builds on ADMM with a pre-inverted penalty library; via **constraint-aware operators, fused single/dual-kernel iterations + CUDA Graph replay, calibration- and measurement-driven auto-configuration, penalty probing and lazy matrix updates**, it cuts latency vs. existing GPU solvers on random QP, random LMPC, humanoid and quadrotor MPC while keeping accuracy and control quality.

1.5–4.3×

Random Dense QP

vs. ReLU-QP (n=220–1400, same accuracy)

3.2–7.3×

Random-System LMPC

vs. ReLU-QP, 0.66–4.23 ms per step

N 10→20

Unitree G1 Push Recovery

Horizon doubled within a 2 ms control period

4.7×

Quadrotor Koopman MPC

vs. CuPIQP, P99 17.6 ms < 20 ms

MPC solves a QP every cycle: latency sets rate and horizon

Interior Point (IP)

HPIPM • PIQP • Clarabel

+ Few iterations
− Large linear solve per step; hard to warm-start

Active Set (AS)

qpOASES • DAQP

+ Efficient warm starts
− Worst case grows combinatorially with constraints

Operator Splitting (ADMM)

OSQP • ProxQP

+ Easy to warm-start and parallelize
− Slower convergence; suits moderate accuracy

↑ FAME-QP's approach

Why GPU-Native

- World models, latent linear dynamics and Koopman models yield large-scale MPC
- Robot simulation, inference and learning already run on GPU
- CPU solvers ⇒ frequent CPU↔GPU transfers, adding latency

Gaps in closest work ReLU-QP → FAME-QP's remedies

- Dense $(n+2m)^2$ operator too large → **smaller constraint-aware operator**
- Many kernel launches per iteration → **fused kernels + CUDA Graph replay**
- Offline precomputation; no online matrix changes → **lazy operator updates**

Each ADMM iteration = 2 mat-vec products + elementwise ops

Condensed LMPC

Eliminate predicted states via dynamics $X = \mathcal{A}\hat{x} + \mathcal{B}U$, keeping only inputs:

$\mathbf{z = U}$, $\mathbf{n = N \cdot n_u}$

$P = \mathcal{B}^{\intercal}\mathcal{B} + \mathcal{R}$, $q = \mathcal{B}^{\intercal}\mathcal{Q}\mathcal{A}\hat{x}$

$G = [\mathcal{B}\ ; I]$, $m = N(n_x+n_u)$

→ Dense matrices, suited to GPU dense linear algebra

Preconditioning

- Modified Ruiz equilibration (10 passes)
- Penalty matrix $\boldsymbol{\rho} = \rho \cdot \mathrm{diag}(b)$, $b = 10^3$ for equality rows
- LQR pre-stabilization advised for open-loop unstable systems

① $\tilde{z} = M \cdot r$
$M = (P+\sigma I+G^{s}\boldsymbol{\rho}G)^{-1}$ pre-inverted

↓

② $\tilde{s} = G \cdot \tilde{z}$
Matrix-vector product

↓

③ $z \leftarrow \alpha\tilde{z} + (1-\alpha)z$
Relaxation ($\alpha = 1.6$)

↓

④ $s = \Pi_C(\zeta)$
$\zeta = \alpha\tilde{s}+(1-\alpha)s+\boldsymbol{\rho}^{-1}\lambda$, clipped elementwise to [l, h]

↓

⑤ $\lambda = \rho(\zeta - s)$
Dual update; build next r

Coupled
Mat-vec products
⇒ bandwidth-bound

Elementwise
Can be fused into the mat-vec kernel

Contribution ①: Constraint-Structure-Aware Operators

Only dense constraint rows enter the operator: up to 4× less storage

Box

G = D = diag(d)

Source: input constraints only

Store M and d: $n^2 + n$ entries

(dense handling needs $3n^2$–$4n^2$)

Split

G = [D ; C]

Source: input + state constraints

D absorbed elementwise into r;

only C (m_c rows) treated as dense

General

Fully dense rows, C = G

Source: input constraints coupled by LQR pre-stabilization

All rows treated as dense,

$m_c = m$

Trade-off between two operator forms (Split / General)

Stacked form $W = [M, MC^T ; CM, CMC^T]$: one mat-vec yields both $\tilde{z}$ and $\tilde{s}$ → 1 kernel

Factored form $F = [M, MC^T]$ plus C: first $\tilde{z}$, then $C\tilde{z}$ → 2 kernels

$(n+m_c)^2 - n(n+2m_c) = \mathbf{m_c^2}$ ← elements saved per iteration by factored form

Ablation (G = [I ; C])

Storage ↓ **1.67–4.00×**

Iteration time ↓ **1.07–3.03×**

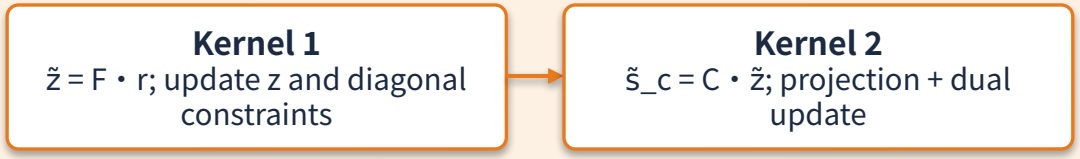
n=512, m_c=128: 10.1 → 3.12 MiB (ReLU-QP: 24.5 MiB)

Contribution ②: Fused Execution + CUDA Graph + Auto-Selection

Iterations fused into 1–2 kernels; one CPU sync per 10 iterations

Single-kernel mode (stacked operator W)

Dual-kernel mode (factored operators F, C)



CUDA Graph Replay

- Capture j_chk = 10 iterations + termination check (Pz, Gz, $G^T\lambda$, two reduction kernels)
- Host reads only 5 scalars per replay: converged / stalled / switch ρ

Auto-selection: model prunes, timing decides

$$t_{iter} \approx n_{ker} \cdot \tau + t_{stream}(b_{op})$$

τ: per-graph-node overhead; t_stream: operator read time (calibrated over 0.25–64 MB, cached per device)

1. Exclude forms that don't fit in memory (penalty library > 60% of free VRAM)
2. Prune forms predicted > 1.5× the best; under constant bandwidth, stacked is faster if $m_c < \sqrt{(\tau \cdot BW/\beta)}$
3. Time placeholder operators at actual sizes, also tuning tile config
4. Choose dual-kernel only if > 10% faster than single-kernel

Contribution ③: Penalty Library, Probing & Online Updates

ρ probing picks the start; lazy updates handle time-varying matrices

Penalty Library

- Pre-invert 5 ρ values, spaced 10×, initial center $\rho_0 = 0.1$
- Residual balancing: $\rho_{new} = \rho \cdot \sqrt{(r_{prim} \cdot \eta_d / (r_{dual} \cdot \eta_p))}$
- Switch to nearest entry only if off by > 5×; no re-inversion
- λ kept on switch (optimum independent of ρ)

Initial Penalty Probing

- Setup: each entry runs ⩽ 400 iters from cold
- Compare time to tightest tol. (1/100, 1/10, 1×)
- Best at edge → recenter, reprobe (max 2)
- Split: also probe $\gamma \in \{10^{-3} \ldots 10\}$ to balance dense rows

Online Updates & Precision

- New q, l, h: reuse scaling, rebuild r
- New P, G: lazy mode recomputes active entry, marks rest stale
- Warm start: may shift prior solution one horizon step
- Auto precision: κ(K) picks fp32 / fp64

Cost: probing runs only at setup, not during solves; in ablations, setup cost is recouped in ~36–122 solves. The penalty grid is fixed within a single solve.

Exp. 1: Cold-Start Solves on Dense Random QPs

Beats all GPU baselines at equal accuracy; all CPU solvers for n ⩾ 220

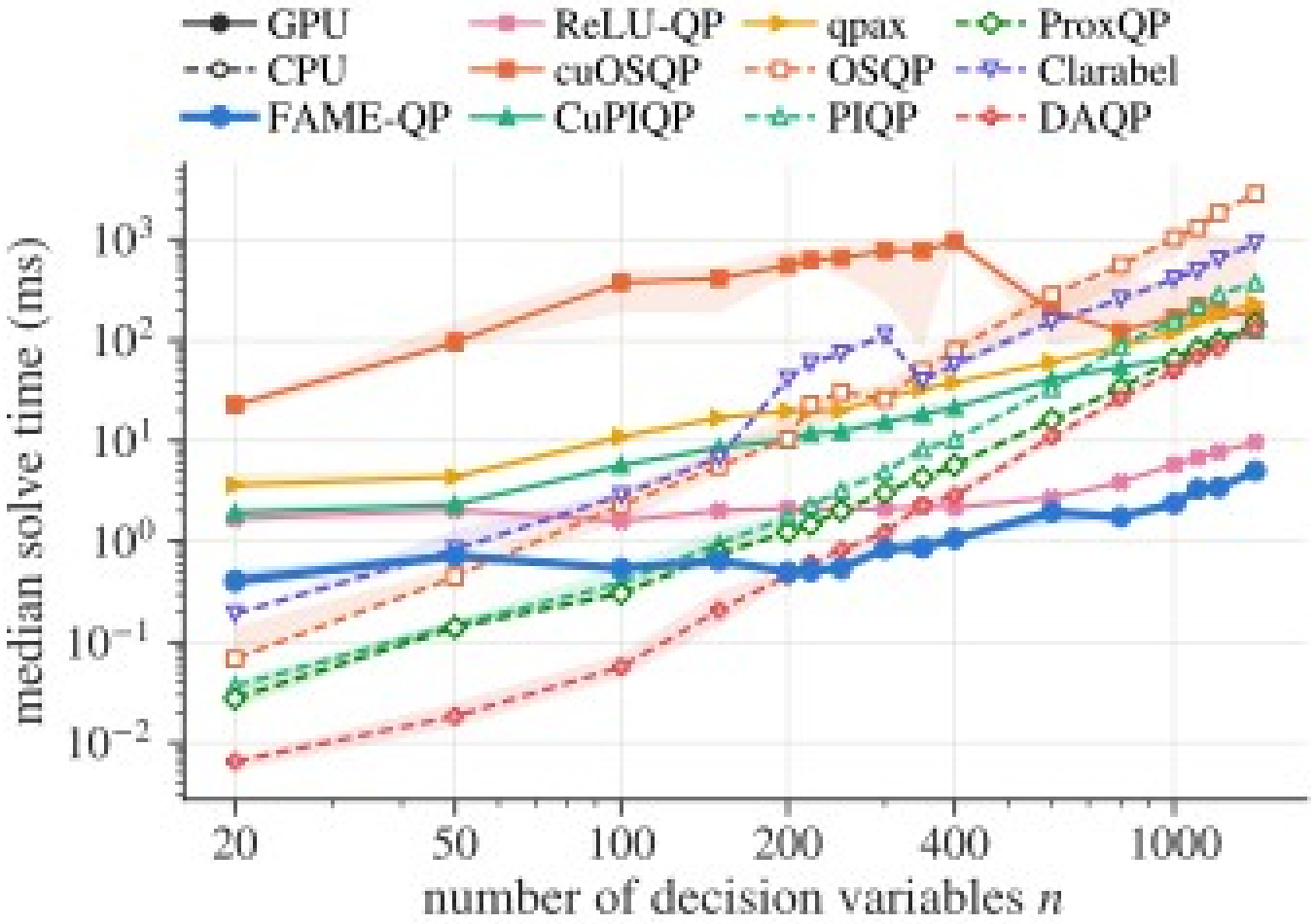


Fig. 1: Equal-accuracy cold-start solve time (solid GPU, dashed CPU; cuOSQP float32, rest float64)

1.5–4.3× vs ReLU-QP
Strongest GPU baseline, n = 220–1400

1.2–25.9× vs DAQP
Strongest CPU baseline, n = 220–1400

Mean solve time at n = 1400

FAME-QP float32 **1.95 ms**

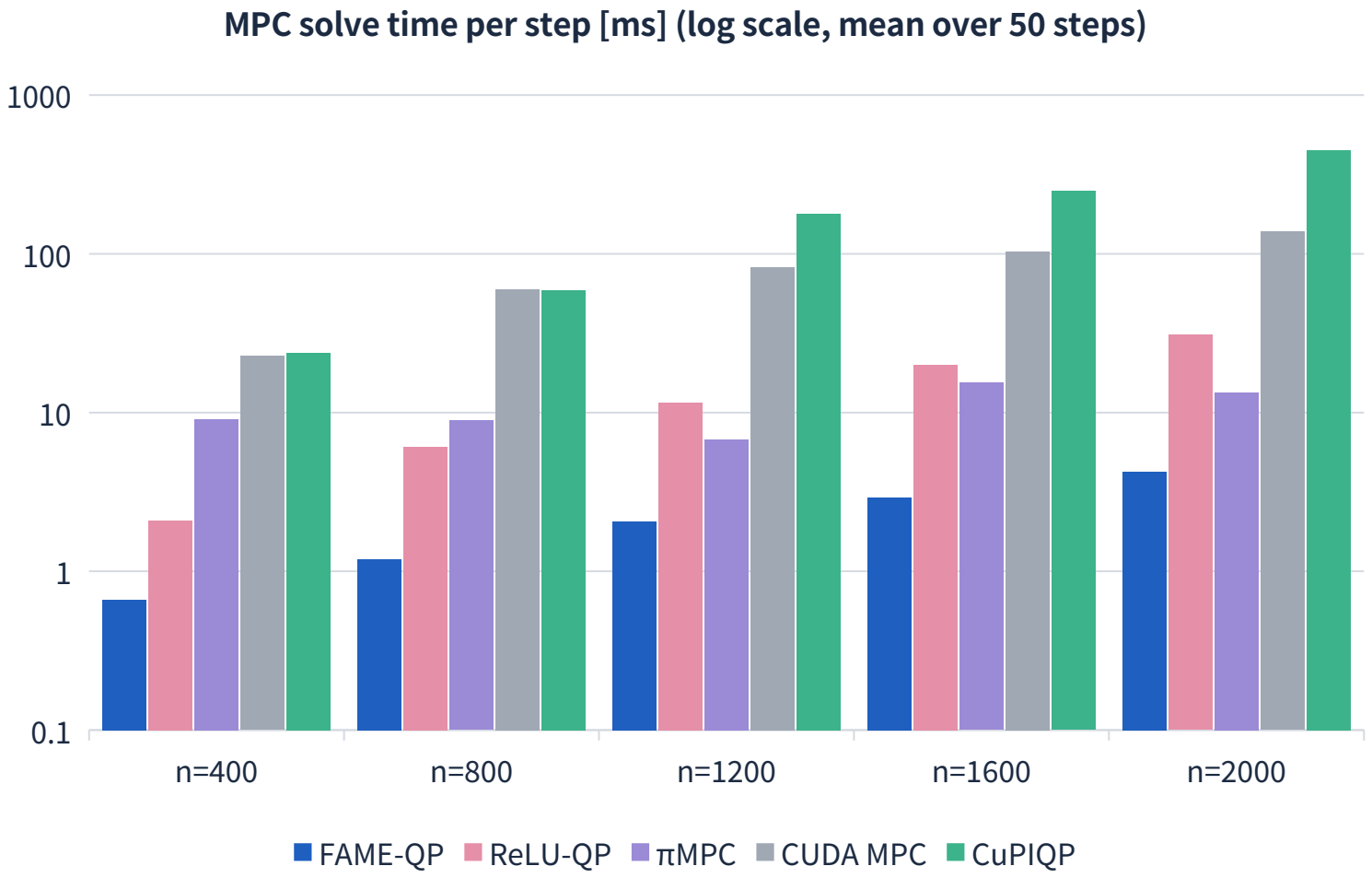
FAME-QP float64 **5.01 ms**

cuOSQP float32 **326 ms**

Note: for small n, CPU solvers such as DAQP are faster; FAME-QP has ~0.4–0.6 ms fixed overhead

Exp. 2: GPU LMPC on Random Unstable Systems (N = 40)

Closed-loop LMPC: fastest at every size, with the lowest timing variance



3.2–7.3×
vs. ReLU-QP (closest QP baseline)

3.2–13.8×
vs. πMPC (fastest MPC baseline)

Accuracy comparison (stopping criteria differ)

- Closed-loop cost: all methods within 0.06%
- Max constraint violation: FAME-QP $\leqslant 2.0\times10^{-3}$; ReLU-QP $1.4–4.3\times10^{-2}$; πMPC $\approx 10^{-4}$; CuPIQP, CUDA MPC $< 10^{-6}$
- πMPC std. dev. exceeds its mean (e.g., 9.06 ± 12.39 ms)

Exp. 3: Humanoid Push Recovery (MuJoCo)

Unitree G1: horizon doubles from 10 to 20 within a 2 ms control period

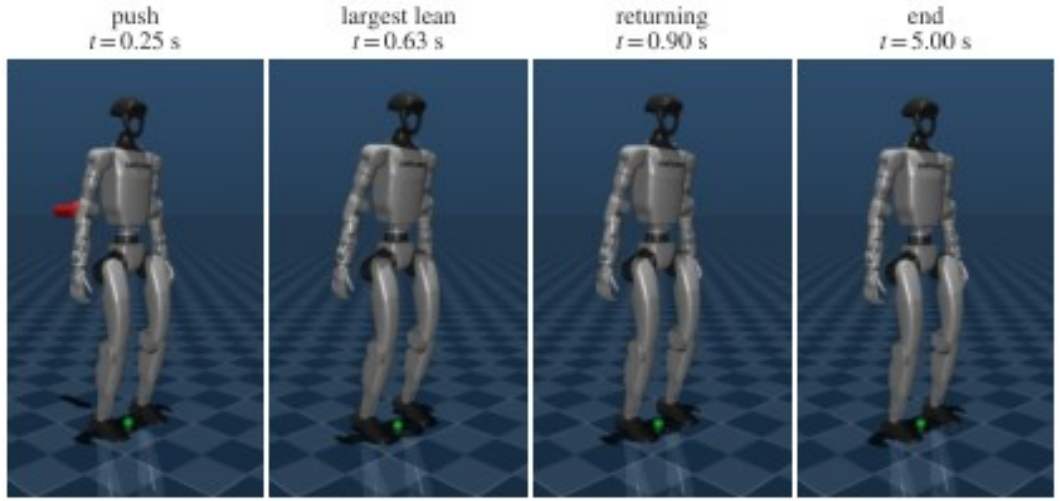
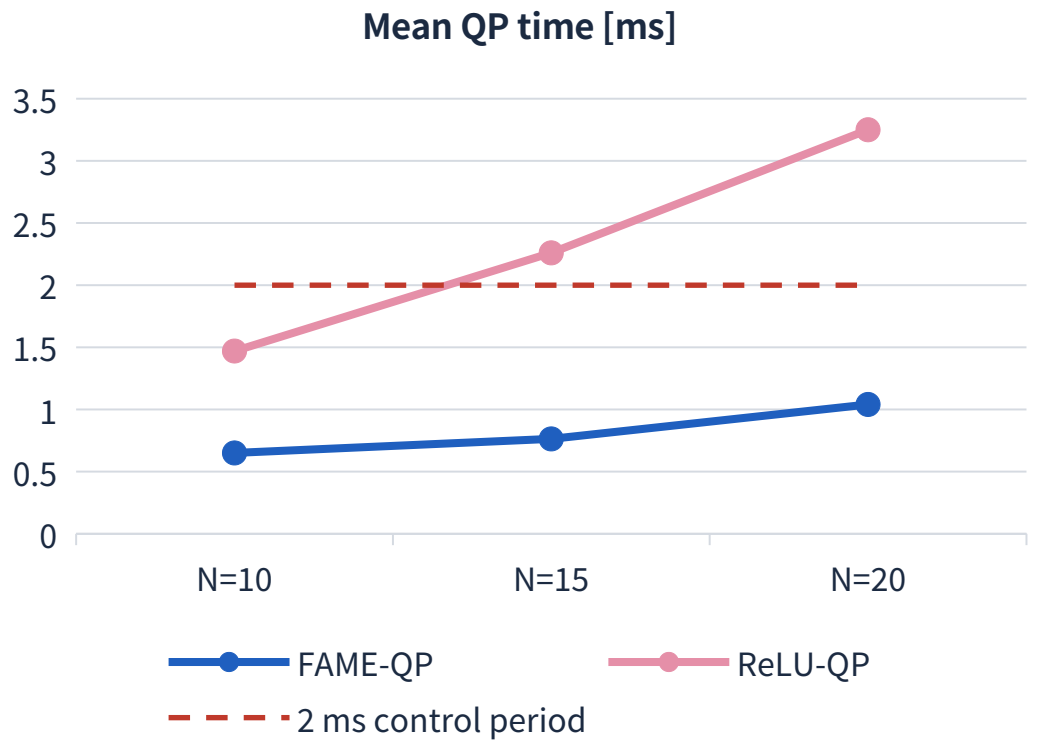


Fig. 2: push → max tilt → recovery → final stance

29 joints • 70-D state • 80 N push for 0.1 s • 5 s recovery • n = m = 29N

N (n)	FAME-QP [ms]	ReLU-QP [ms]	CoM RMS [mm]
10 (290)	**0.650 ± 0.091**	1.47 ± 0.05	18.7 / 18.7
15 (435)	**0.763 ± 0.116**	2.26 ± 0.12	19.1 / 19.2
20 (580)	**1.04 ± 0.20**	3.25 ± 0.13	20.5 / 19.5



QP time ↓ 2.3–3.1×; ReLU-QP exceeds 2 ms for N ⩾ 15; both recover balance comparably

Exp. 4: Skydio X2 Quadrotor Koopman-LMPC (MuJoCo)

QP matrices change per step: 4.7× speedup; P99 < control period

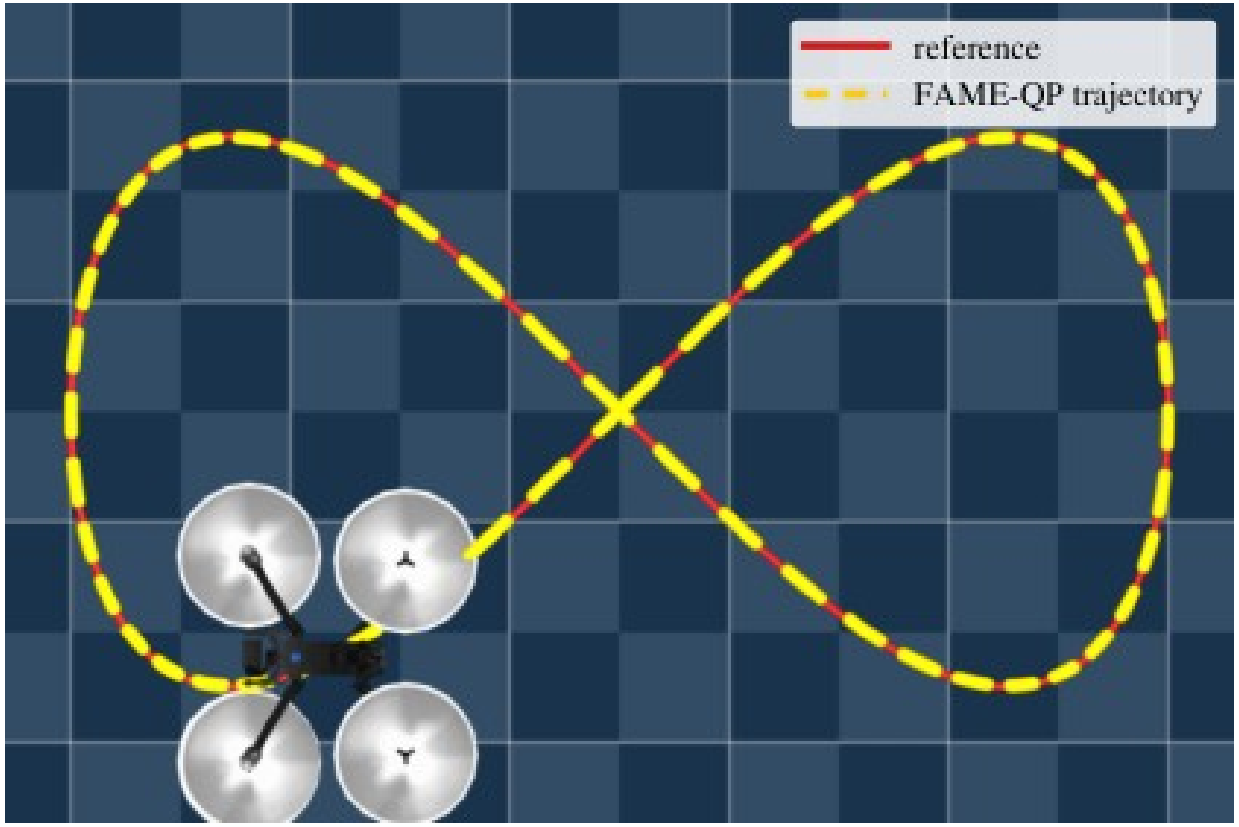
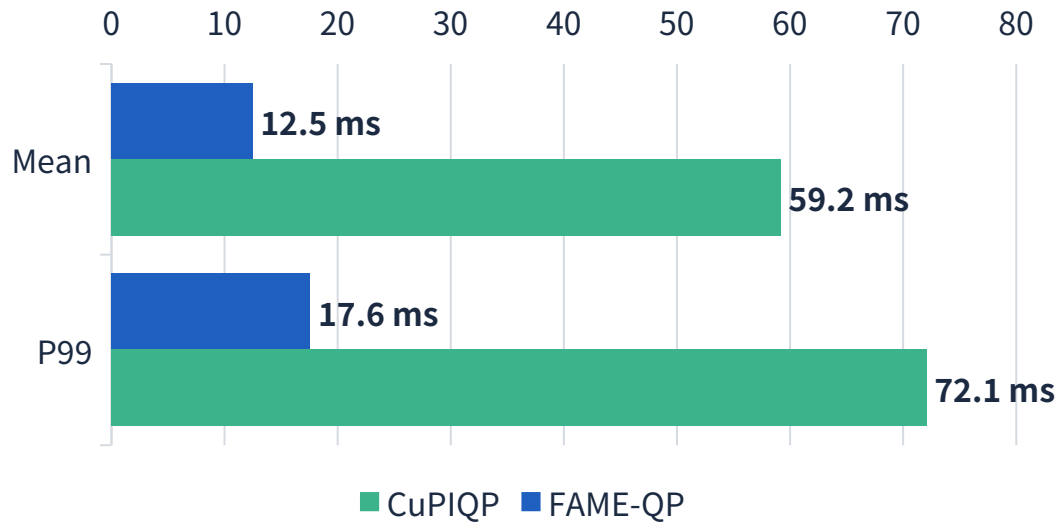


Fig. 3: lemniscate reference (red) vs. FAME-QP trajectory (yellow dashed)

50 Hz • 4 inputs • N = 70 (1.4 s) • n = 280 • 20 s tracking



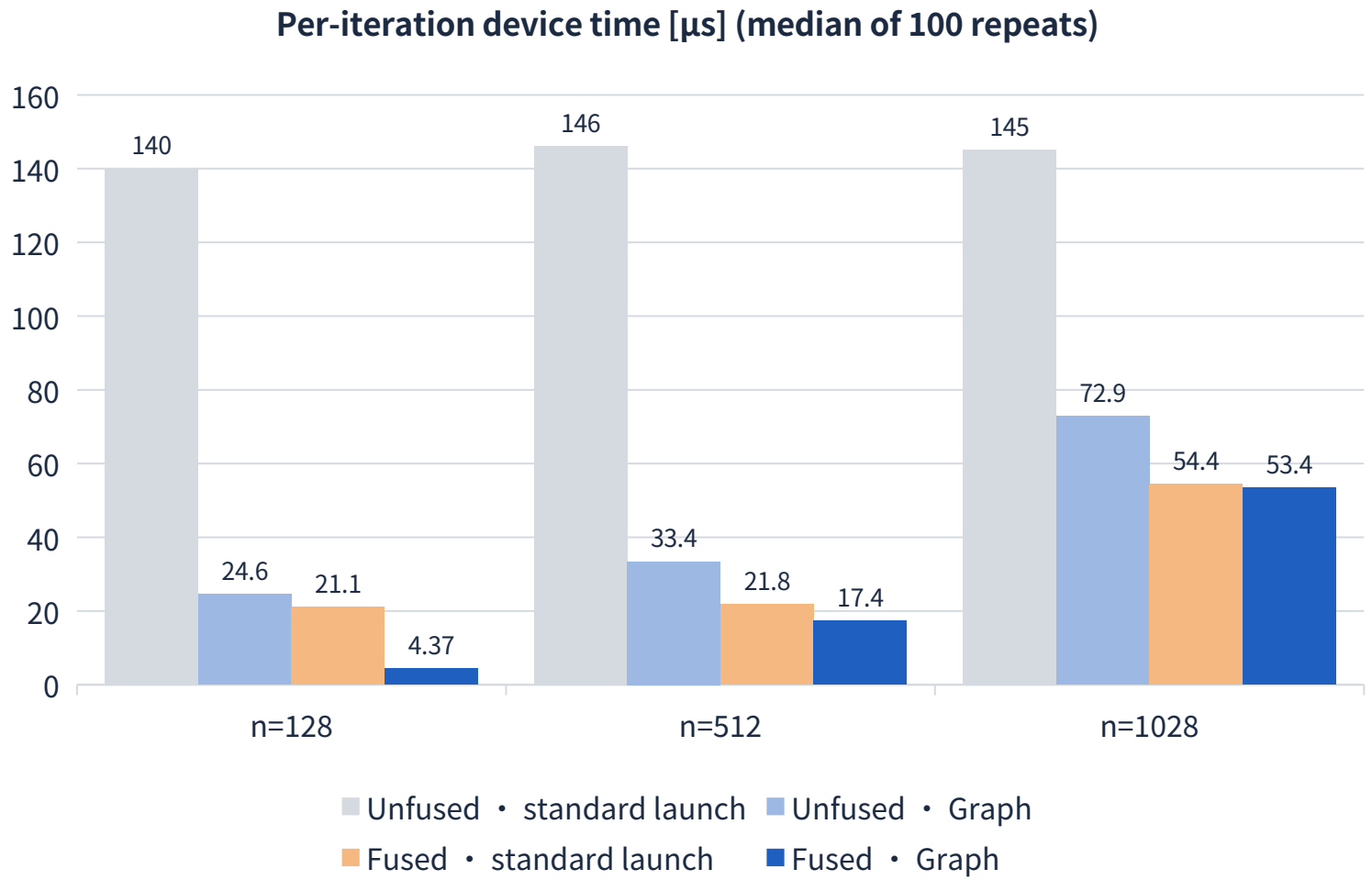
4.7×
Mean QP time speedup

17.6 ms
P99 < 20 ms control period

2.7 mm
Position RMSE (CuPIQP 3.4 mm)

Ablation 1: GPU Execution

Fusion & Graph replay compound: small-n iteration 140 µs → 4.37 µs



32×

n=128: 140 → 4.37 µs; 14 kernels merged into 1

How to Read This Chart

- Graph replay removes host launch overhead; fusion cuts kernel count and memory round trips; effects stack
- Larger problems are dominated by mat-vec (bandwidth), so gains shrink: 72.9 → 53.4 µs at n=1028
- All four implementations give identical iterates (error $\leqslant 3\times10^{-10}$)

Probing cuts iterations; auto-selection & lazy updates preserve speed

Initial penalty probing (dense random QP, $\varepsilon = 10^{-5}$)

n	Iters: default	Iters: probed	Time: default	Time: probed
250	74	46	1.10 ms	0.79 ms
600	98	48	3.78 ms	1.88 ms
1000	128	40	10.6 ms	3.21 ms

Key Findings

- Fusion × probing gains multiply: **2.8–4.2×** (fusion 2.00→1.26×, probing 1.40→3.35×)
- To reach 10^{-3} accuracy, FAME-QP needs 30–38 iterations vs. 30–32 for ReLU-QP

⇒ The speedup over ReLU-QP **comes mainly from cheaper iterations**, not fewer of them

Automatic Operator Selection

Picked the faster form at 30 of 36 test points; worst miss 9.5% slower (within the 10% retention threshold). Fixed stacked-only or factored-only: up to 2.2× / 1.5× slower. Model-only estimates misplace the crossover by ~2×.

γ Probing for Split Structure

Quadrotor KQ-LMPC replay (N=50): probing ρ lowers median and tail step times; adding γ probing leaves the median nearly unchanged, further shortens the tail, and nearly eliminates in-solve penalty switches.

Lazy Matrix Updates

With full refresh, recomputing 5 library entries dominates per-step cost; lazy refresh roughly halves update and median step times, leaving iterations, solutions and tracking error unchanged.

Verdict: solid GPU engineering, scoped to medium-scale dense MPC

Strengths

✓ Changes execution, not the algorithm: overheads dissected one by one, each design backed by ablations

✓ Model pruning + measured choice: paper shows model-only crossover estimates are off by ~2×

✓ Supports online matrix updates, suiting Koopman / linearized time-varying MPC

✓ Equal-accuracy comparisons, reporting constraint violation, P99 and setup payback; open-source code

Limitations & Questions

! Dense QPs only (all experiments condensed): storage grows as n^2, and the penalty library stores 5 copies (sparse form left to future work)

! Slower than CPU solvers like DAQP for $n < 200$; ~0.4 ms fixed GPU overhead

! ADMM accuracy is capped: constraint violation ~10^{-3}; float32 stalls near 10^{-6}

! Setup probing needs 36–122 solves to pay off

! Only one laptop GPU tested (RTX 3080 Laptop); CUDA MPC reimplemented by the authors; G1 reports mean times only

Key Takeaways

Problem	Low-latency solving of dense QPs from condensed linear MPC within GPU-native pipelines
Approach	ADMM + pre-inverted penalty library; constraint-aware operators; fused single/dual-kernel iterations + CUDA Graph; calibration- and measurement-driven auto-config; penalty probing; lazy updates
Results	1.5–7.3× faster than the strongest GPU baseline ReLU-QP (random QP / LMPC); G1 horizon 10 → 20 within a 2 ms period; quadrotor P99 stays below the control period despite time-varying matrices
Source of Speedup	Mainly cheaper iterations, not fewer; best for repeated same-shape solves, moderate accuracy and latency-critical settings

arXiv 2611.13766 • Code: github.com/santoshrajkumar/fame-qp